TangleForensix — Complete Project Documentation

Table of Contents

---

1. Project Overview

TangleForensix is a forensic analysis tool for Bitcoin (BTC) transactions. It reads a CSV file of raw Bitcoin transactions, applies a series of graph-based heuristics to detect suspicious patterns, clusters addresses that likely belong to the same real-world entity, and renders an interactive browser-based GUI for visual exploration.

What problems does it solve?

Bitcoin is pseudonymous — every wallet address is just a random string. However, because the blockchain is public, the flow of funds between addresses can be analysed. TangleForensix automates the following forensic tasks:

Task	What it finds
Change address detection	Which output of a transaction is the "change" going back to the sender
Address clustering	Which addresses are owned by the same person/entity
Sweep detection	Consolidation of many inputs into one output (wallet draining)
Wash trading detection	Funds that never leave the sender's own cluster
Peeling chain detection	Sequential fund obfuscation pattern

---

2. Architecture & Pipeline

```
btc_transactions.csv
        |
        ▼
load_transactions_csv()     ← reads & cleans CSV, derives DAG parents
        |
        ▼
detect_peeling_chains()     ← pre-scan for peeling chain candidates
        |
        ▼
cluster_and_analyze()       ← runs Algorithms 1, 2, 3, peeling, sweep
        |                        builds Union-Find (DSU) clusters
        ▼
detect_wash_trading_pass()  ← second pass using completed DSU
```

```
        |
        ▼
write_results_json()        ← saves btc_results.json
        |
        ▼
build_html()                ← embeds JSON into self-contained HTML page
        |
        ▼
btc_tangleforensix.html     ← open in browser
```

Module responsibilities

Module	Role
`tangle_heuristic_mapper.py`	All analysis logic: heuristics, clustering, DSU
`tangle_renderer.py`	Renders interactive HTML/D3.js GUI
`main.py`	Thin orchestrator: calls mapper then renderer

---

3. Data Format

Input CSV (BTC mode)

The tool auto-detects BTC format when the column `transaction_id` is present.

```
transaction_id, input_addresses,     input_values, output_addresses,       output_values
tx_abc123,      addr1;addr2,         5000;3000,    addr3;addr4,            7500;200
tx_def456,      addr3,               7500,         addr5;addr6,            6000;1400
```

- Multiple addresses/values in one field are separated by `;`
- Addresses named `unknown`, empty strings, or with value `0` are stripped (they represent OP_RETURN or unspendable outputs)

Output JSON

```json
{
  "transactions": [
    {
      "id": 1,
      "flags": ["large_value_examiner"],
      "change_address": "addr3",
      "detected_by": "large_value_examiner",
      "inputs":  [{"addr": "addr1", "val": 5000}, {"addr": "addr2", "val": 3000}],
      "outputs": [{"addr": "addr3", "val": 7500}, {"addr": "addr4", "val": 200}]
    }
  ],
  "clusters": [
    {"id": 0, "addresses": ["addr1", "addr2", "addr3"]}
  ]
}
```

---

4. Core Data Structures

Transaction dataclass

```python
@dataclass
```

```
class Transaction:
    id: int
    type: str
    timestamp: str
    parents: List[int]          # derived from UTXO flow
    input_addresses:  List[str]
    input_values:     List[int]
    output_addresses: List[str]
    output_values:    List[int]
```

DisjointSet (Union-Find)

Used to group addresses that belong to the same entity. Two key operations:

- **`union(a, b)`** — merge the clusters of address `a` and address `b`
- **`find_root(x)`** — return the representative ("root") of x's cluster

```python
class DisjointSet:
    parent: Dict[str, str]   # each address points to its parent
    rank:   Dict[str, int]   # used for balanced merging
```

Path compression makes repeated `find_root` calls almost O(1).

---

5. Algorithm 1 — Excess Input Filter

What it detects

When a transaction has **more than 1 input** and exactly **2 outputs**, and one output value is smaller than the smallest input value — that small output is very likely the **change** going back to the sender.

Intuition

Imagine you have two $50 bills and you want to pay $85. You hand over both bills and get $15 change back. The $15 change is identifiably "yours" because it's smaller than the smallest bill you used ($50).

Formal rule

```
Pre-condition: n > 1 AND m == 2
For each output j:
    if output_value[j] < min(input_values):
        → output_address[j] is the change address
```

Example

```
Transaction T:
  Inputs:   [addr_A: 8000 sat, addr_B: 5000 sat]
  Outputs:  [addr_C: 12500 sat, addr_D: 200 sat]

min(input_values) = 5000

Output addr_D = 200 sat  <  5000   ✓  → addr_D is change
```

Result: Flag = `excess_input_filter`, change = `addr_D`

Clustering: `addr_D` (change) is merged with `addr_A` (first input) in the DSU, because both belong to the sender.

Code

```python
def excess_input_filter(tx):
    n, m = len(tx.input_addresses), len(tx.output_addresses)
    if n > 1 and m == 2:
        I_min = min(tx.input_values)
        for j in range(m):
            if tx.output_values[j] < I_min:
                return (1, tx.output_addresses[j])
    return (0, "")
```

---

6. Algorithm 2 — Large Value Examiner

What it detects

When one output is so much larger than all other outputs combined that it "dominates" — that dominant output is likely the **change** address (the sender keeping most of the value and sending only a small payment).

Intuition

You have 10,000 sat and want to pay a friend 500 sat. You create a transaction:
- Output 1: 9,450 sat → your change address
- Output 2: 500 sat → friend's address

The 9,450 sat output absolutely dominates. Algorithm 2 catches this.

Formal rule

```
Sort outputs descending by value.
Let Omax = largest output value, A_max = its address.
Let Oothers = sum of all other output values.

discrepancy_ratio = (Omax - Oothers) / Omax

If discrepancy_ratio > threshold (0.85):
    → A_max is the change address

Special case: if A_max is already one of the INPUT addresses
    → skip (this is just address reuse, not new evidence)
```

Threshold = 0.85 (from paper)

This means the largest output must account for more than 92.5% of total output value:
```
(Omax - Oothers) / Omax > 0.85
Omax - Oothers > 0.85 * Omax
Oothers < 0.15 * Omax
```
So the payment is less than 15% of the change.

Example

```
Transaction T:
  Outputs: [addr_X: 9000 sat, addr_Y: 500 sat]
```

```
Omax = 9000 (addr_X)
Oothers = 500

ratio = (9000 - 500) / 9000 = 8500 / 9000 = 0.944  >  0.85  ✓

→ addr_X is the change address
```

Result: Flag = `large_value_examiner`, change = `addr_X`

Counter-example (address reuse case)

```
Transaction T:
  Inputs:  [addr_X: 9500 sat]
  Outputs: [addr_X: 9000 sat, addr_Y: 500 sat]

Omax address = addr_X → BUT addr_X is also an INPUT address.
→ This is just the sender paying themselves (address reuse).
→ Return (0, "") — no new clustering evidence.
```

Code

```python
def large_value_examiner(tx, t=0.85):
    paired = sorted(zip(tx.output_values, tx.output_addresses), key=lambda p: -p[0])
    O_max_val, A_max = paired[0]
    O_others = sum(v for v, _ in paired[1:])

    if O_max_val == 0:
        return (0, "")
    ratio = (O_max_val - O_others) / O_max_val
    if ratio > t:
        if A_max in set(tx.input_addresses):
            return (0, "")       # address reuse, skip
        return (1, A_max)
    return (0, "")
```

---

7. Algorithm 3 — First Time Recipient

What it detects

When exactly **one output address has never been seen before** (brand new to the blockchain history seen so far), that new address is the **payment** going to a new recipient — and all other output addresses (already seen) are the change going back to the sender.

Intuition

You always reuse your own wallet addresses (they are already "known"). When you pay someone new, their address appears for the first time. So: if only one output is a fresh address and all others are old/known, that fresh address is the payment and the old ones are your change.

Wait — that's inverted. In this algorithm:
- New (unseen) address = the **payment** recipient (a new person)
- Known (already seen) addresses = the **change** going back to sender

So the **change address** is the one that IS already in history H.

Actually, re-reading: the function returns the address that is NOT in H and NOT in inputs —

this is the new/fresh address. But in the context of the heuristic, the "change_address" returned here is the fresh output (`o'`). The reasoning is: the fresh address is the payment recipient, and the non-fresh outputs are the change. The function returns `o'` (the fresh one) as the detected special address.

How it works

H = set of all addresses seen in previous transactions (historical set)

```
new_outputs = [o for o in outputs if o ∉ H AND o ∉ inputs]

Condition 1: len(new_outputs) == 1   (exactly one brand-new output)
Condition 2: every OTHER output is already in H (known addresses)

If both conditions met:
    → o' = new_outputs[0]
    → flag this transaction
```

Critical implementation detail

H is updated **after** the heuristic runs on each transaction. So H contains history from all *previous* transactions but NOT the current one. This prevents the current tx's own outputs from "poisoning" the freshness check.

Example

```
History H = {addr_A, addr_B, addr_C, addr_D}  (seen in past txs)

Transaction T:
  Inputs:  [addr_A: 10000]
  Outputs: [addr_B: 9000, addr_NEW: 800]

new_outputs (not in H, not in inputs):
  addr_B  → in H ✗ (already known)
  addr_NEW → NOT in H ✓, NOT in inputs ✓  → new_outputs = [addr_NEW]

len(new_outputs) == 1  ✓
Every other output (addr_B) is in H  ✓

→ addr_NEW is flagged (the payment recipient)
→ addr_B is the change (known address = sender's own)
```

Result: Flag = `first_time_recipient`, the returned "change" address = `addr_NEW`

Note: in the clustering step, `addr_NEW` gets unioned with the input `addr_A`. This is because the caller treats the returned address as change (something connected to the sender), even though semantically it's a payment. This is a design choice from the original paper.

Code

```python
def first_time_recipient(tx, H):
    inputs_set = set(tx.input_addresses)
    new_outputs = [o for o in tx.output_addresses
                   if o not in H and o not in inputs_set]

    if len(new_outputs) != 1:
        return (0, "")

    o_prime = new_outputs[0]
```

```
    for p in tx.output_addresses:
        if p == o_prime:
            continue
        if p not in H:
            return (0, "")       # another unknown output found → condition fails

    return (1, o_prime)
```

---

8. Algorithm 4 — Address Clustering (DSU)

What it does

Algorithm 4 is the **backbone** of the entire system. It takes the results of Algorithms 1, 2, 3 and builds a **Union-Find (Disjoint Set Union)** structure that groups addresses belonging to the same real-world entity.

Two clustering rules

Rule A — Change address ownership:
If a change address `c` is detected for a transaction with input `addr_0`:
```
union(c, addr_0)
```
The change goes back to the sender, so c and addr_0 belong to the same person.

Rule B — Common input ownership (co-spend heuristic):
If a transaction has multiple inputs `[addr_0, addr_1, addr_2, ...]`:
```
union(addr_0, addr_1)
union(addr_0, addr_2)
:::
```
To authorise a spend, the owner must sign with ALL input private keys — so all inputs must be controlled by the same entity.

How DSU works (step by step)

Initial state — each address is its own cluster:
```
addr_A → {addr_A}
addr_B → {addr_B}
addr_C → {addr_C}
addr_D → {addr_D}
```

After `union(addr_A, addr_B)`:
```
{addr_A, addr_B}    addr_C    addr_D
```

After `union(addr_B, addr_C)` (B and C):
```
{addr_A, addr_B, addr_C}    addr_D
```

Even though we said "B and C", since A and B are already merged, all three end up together.

Full algorithm flow

```
1. Initialize DSU with ALL addresses from ALL transactions.
2. Pre-compute peeling chain map.
```

```
3. For each transaction T in timestamp order:
   a. Run Algorithms 1, 2, 3, peeling chain on T
   b. If any heuristic found a change address c:
      → union(c, T.input_addresses[0])
   c. If T has multiple inputs:
      → union(T.input_addresses[0], T.input_addresses[i])  for all i
   d. Add all of T's addresses to history H
4. Build cluster dictionary: group all addresses by their DSU root.
```

Example walkthrough

```
Transactions:
  T1: inputs=[A, B], outputs=[C: 9000, D: 200]
      → excess_input_filter fires → change = D
      → union(D, A)   (change + first input)
      → union(A, B)   (co-spend: both inputs)
      Clusters so far: {A, B, D}  {C}

  T2: inputs=[C], outputs=[E: 8500, F: 400]
      → excess_input_filter fires → change = F
      → union(F, C)
      Clusters so far: {A, B, D}  {C, F}  {E}

  T3: inputs=[A, E], outputs=[G: 17000]
      → sweep (1 output) → no change detected
      → union(A, E)   (co-spend)
      Clusters so far: {A, B, D, E}  {C, F}  {G}
                        (E merged with A's cluster)
```

Final clusters reveal that addresses A, B, D, E are all controlled by the same entity.

---

9. Sweep Detection

What it detects

A **sweep transaction** consolidates ALL funds from multiple input addresses into a single output — typically used when someone is "draining" a wallet or moving all funds to a new address.

Rule

```
IsSweep(T) = 1  iff  |inputs| > 1  AND  |outputs| == 1
```

Example

```
Transaction T:
  Inputs:  [addr_A: 3000, addr_B: 5000, addr_C: 2000]
  Outputs: [addr_NEW: 9800]    ← single output (200 sat = fee)

|inputs| = 3 > 1  ✓
|outputs| = 1     ✓

→ Sweep detected
```

Why it matters: Sweeps strongly indicate the same entity controls all inputs. It's also a strong clustering signal (all inputs → same owner).

Note: Sweep takes priority over wash trading — if a tx is flagged sweep, it's NOT also checked for wash trading to avoid double-counting.

---

10. Wash Trading Detection

What it detects

A **wash trading** transaction is one where ALL output addresses resolve to the same ownership cluster as the input addresses — meaning the funds never actually leave the sender's control. This is used to create fake trading volume or obscure trails.

Rule (post-clustering)

```
IsWashTrading(T) = 1  iff:
  - T has at least 1 input
  - T has at least 2 outputs  (1-output self-sends are just transfers)
  - For EVERY output address o:
      find_root(o) ∈ { find_root(i) for i in inputs }
```

Why post-clustering?

Wash trading can only be detected AFTER the DSU is fully built (after `cluster_and_analyze` completes), because we need to know which addresses share a cluster. That's why it runs in a separate second pass — `detect_wash_trading_pass()`.

Example

```
After clustering, cluster roots:
  addr_A → root_1
  addr_B → root_1    (same cluster as A)
  addr_C → root_1    (same cluster)
  addr_D → root_2    (different entity)

Transaction T:
  Inputs:  [addr_A]
  Outputs: [addr_B: 5000, addr_C: 4000]

Sender root = find_root(addr_A) = root_1

find_root(addr_B) = root_1  ✓ (same as sender)
find_root(addr_C) = root_1  ✓ (same as sender)

All outputs stay in sender's cluster → Wash trading!
```

Counter-example (not wash trading)

```
Transaction T:
  Inputs:  [addr_A]
  Outputs: [addr_B: 5000, addr_D: 4000]

find_root(addr_D) = root_2  ≠  root_1

→ addr_D goes to a different entity → NOT wash trading
```

Priority rules (no double-flagging)

- If `sweep` is already flagged → skip wash trading check
- If `peeling_chain` is already flagged → skip wash trading check

---

11. Peeling Chain Detection

What it detects

A **peeling chain** is a Bitcoin obfuscation technique where funds are passed through a series of transactions, each "peeling off" a small payment while the bulk (the remainder/change) moves on to the next transaction. This creates a long chain that is hard to follow manually.

```
Tx1: [10000 sat input] → [200 sat payment] + [9700 sat remainder]
Tx2: [9700 sat input]  → [200 sat payment] + [9400 sat remainder]
Tx3: [9400 sat input]  → [200 sat payment] + [9100 sat remainder]
:::
```

Shape of a peeling chain hop

Each hop has:
- Exactly **1 input** address
- Exactly **2 outputs**
- The **larger output strictly dominates** the smaller (the remainder is bigger than the payment)
- The larger output (remainder) **is NOT the same address as the input** (no address reuse)
- The remainder address **appears as an input in another transaction** in the dataset (confirming the chain continues)

Detection function

```python
def _is_peel_link(tx):
    if len(tx.input_addresses) != 1 or len(tx.output_addresses) != 2:
        return (0, "")
    # sort outputs largest first
    paired = sorted(zip(tx.output_values, tx.output_addresses), key=lambda p: -p[0])
    (rem_v, rem_a), (peel_v, _) = paired
    if rem_v <= peel_v:     # larger must strictly dominate
        return (0, "")
    return (1, rem_a)       # remainder address
```

Full detection

```python
def detect_peeling_chains(transactions):
    all_input_addrs = {a for tx in transactions for a in tx.input_addresses}

    chain_map = {}
    for tx in transactions:
        ok, rem_a = _is_peel_link(tx)
        if not ok:
            continue
        if rem_a == tx.input_addresses[0]:
            continue    # address reuse — not a real chain hop
        if rem_a in all_input_addrs:
            chain_map[tx.id] = rem_a    # confirmed: remainder is re-spent

    return chain_map
```

Example

```
Transactions:
  T1: input=[W1: 10000], outputs=[addr_pay1: 300, W2: 9600]
      _is_peel_link → rem_a = W2 (larger)
      W2 ≠ W1 (no address reuse)  ✓
      W2 appears as input in T2   ✓
      → T1 flagged as peeling_chain

  T2: input=[W2: 9600], outputs=[addr_pay2: 300, W3: 9200]
      Similar analysis → T2 flagged as peeling_chain
```

Why not just flag all 1-in/2-out transactions?

Many ordinary Bitcoin transactions have 1 input and 2 outputs (payment + change). Simply having this shape is NOT enough — we additionally require that the remainder is provably re-spent in the dataset, confirming it's part of a chain and not just a normal payment.

---

12. GUI — Transaction View

What is shown

Each **node** = one Bitcoin transaction (circle, colored by primary flag).

Each **edge** = UTXO flow: an edge from Tx A → Tx B means Tx B spends an output address that Tx A produced.

Node colors

Color	Flag	Meaning
Blue-grey	normal	No suspicious pattern detected
Yellow	sweep	Multi-input → single-output consolidation
Red	wash_trading	Funds stay within sender's cluster
Orange	peeling_chain	Sequential peel obfuscation hop
Green	excess_input_filter	Change detected by Algorithm 1
Light blue	large_value_examiner	Change detected by Algorithm 2
Purple	first_time_recipient	Change detected by Algorithm 3

Interactive features

- **Hover** over a node → tooltip shows tx ID, flags, input/output count
- **Click** a node → highlights that tx and all its direct neighbours; shows full details in the Output Window (inputs, outputs, change address, flags)
- **Flag filter checkboxes** → show/hide transactions by flag type
- **Address search** → paste any partial address to highlight matching transactions
- **ID range filter** → filter transactions by numeric ID range
- **Zoom controls** → zoom in/out, reset
- **`Esc` key** → clear all selections

---

13. GUI — Cluster View

What is shown

Each **node** = one address cluster. Node **size is proportional** to the number of addresses in that cluster (larger node = more addresses controlled by one entity).

Only clusters with **2 or more addresses** are shown (singleton clusters are hidden to reduce noise).

Interactive features

- **Hover** → shows cluster ID and number of addresses
- **Click** → lists all member addresses in the Output Window (up to 25 shown)
- **Fund flow edges** (toggle button) → draws edges between clusters where funds were transferred
- **Address search** → highlights the cluster containing a searched address
- **Zoom + ID range filter** → same as Transaction View

---

14. Running the Project

Full pipeline

```bash
python main.py
```

Uses default input: `btc_transactions.csv` in current directory.

Custom dataset

```bash
python main.py --input /path/to/your_dataset.csv \
               --json  output/results.json \
               --html  output/gui.html
```

Run analysis only (no GUI)

```bash
python tangle_heuristic_mapper.py \
  --input  /path/to/dataset.csv \
  --output output/results.json
```

Run renderer only (from existing JSON)

```bash
python tangle_renderer.py \
  --input  output/results.json \
  --output output/gui.html
```

Open the GUI

```bash
xdg-open output/btc_tangleforensix.html    # Linux
```

Or paste the file path directly into your browser.

---

15. End-to-End Example

Input (3 transactions)

```
transaction_id, input_addresses, input_values, output_addresses, output_values
tx1,            A;B,             6000;4000,    C;D,             9500;200
tx2,            C,               9500,         E;F,             9000;400
tx3,            E;F,             9000;400,     G,               9300
```

```

### Step 1 — Load

- T1: inputs=[A:6000, B:4000], outputs=[C:9500, D:200]
- T2: inputs=[C:9500], outputs=[E:9000, F:400]
- T3: inputs=[E:9000, F:400], outputs=[G:9300]

### Step 2 — Peeling chain pre-scan

- T2: 1 input (C), 2 outputs (E=larger, F=smaller). rem_a=E. E appears as input in T3. ✓
- T1 and T3: shape doesn't match (T1 has 2 inputs, T3 has 1 output)
- Peeling chain map: {T2: E}

### Step 3 — cluster_and_analyze

**T1:**
- Alg 1: n=2>1, m=2, min(inputs)=4000. D=200 < 4000 → change=D
- Alg 2: ratio=(9500-200)/9500=0.979 > 0.85, but C not in inputs → change=C? No, alg1 fires
first.
- Alg 3: H=empty, new outputs = [C, D], count=2 ≠ 1 → no match
- Peeling: T1 not in peeling map → no
- Sweep: 2 inputs, 2 outputs → no
- **Flag: excess_input_filter, change=D**
- DSU: union(D, A), union(A, B)
- Clusters: {A,B,D}, {C}
- H updated: {A,B,C,D}

**T2:**
- In peeling chain map → change=E
- **Flag: peeling_chain, change=E**
- DSU: union(E, C)
- Clusters: {A,B,D}, {C,E}, {F}
- H updated: {A,B,C,D,E,F}

**T3:**
- Alg 1: n=2>1, m=1 → no (m≠2)
- Sweep: 2 inputs, 1 output → YES
- **Flag: sweep**
- DSU: union(E, F) → {A,B,D}, {C,E,F}, {G}
- H updated: {A,B,C,D,E,F,G}

### Step 4 — Wash trading pass

- T1: sweep? No. wash trading? outputs=[C,D]. find_root(C)=C_root, find_root(D)=A_root.
Different → no.
- T2: peeling_chain → skip.
- T3: sweep → skip.

### Final clusters

| Cluster | Addresses | Meaning |
|---|---|---|
| 0 | {A, B, D} | Same entity: A and B co-spent (T1), D is change back to them |
| 1 | {C, E, F} | Same entity: C received payment (T1), sent on (T2), E=change, F also from
T3 co-spend |
| 2 | {G} | New recipient of the sweep |

### Final flags

| Tx | Flag | Change Address |
|---|---|---|
| T1 | excess_input_filter | D |
| T2 | peeling_chain | E |
| T3 | sweep | — |
```

Documentation generated for TangleForensix — Bitcoin Forensic Analysis Tool
Based on: "TangleForensix: A tool for forensic analysis of IOTA funds", IIIT Allahabad